

MODULE 04 · PART 2 - WINDOW FUNCTIONS

Ranking and Deduplication

Four functions that differ in one respect only: what each of them does when two rows are equal.

PART 2 of 8 MODULE 4 of 26 RUNS about 32 min

All four answer the same question. They disagree only about **what to do with a tie.**

The same eight rows, four functions

UNITS SOLD	ROW_NUMBER, RANK, DENSE_RANK, NTILE(4)
900	1 1 1 1
850	2 2 2 1
850	3 2 2 2
700	4 4 3 2
640	5 5 4 3
640	6 5 4 3
640	7 5 4 4
500	8 8 5 4

THE TAKEAWAY

Read the second and third rows across. Identical inputs, and one function says 2 and 3 while the others say 2 and 2. Nothing else in this module matters more.

Three names, borrowed from sport

- ◆ **Ordinal ranking is ROW_NUMBER.** Everyone gets a distinct position, so somebody has to be picked to go first.
- ◆ **Competition ranking is RANK.** Joint second, and then nobody is third. That is how a race is scored.
- ◆ **Dense ranking is DENSE_RANK.** Joint second, and the next one is third. That is how a league table of scores is read.
- ◆ **The sporting analogy is not decoration.** If you can say which one a race would use, you have chosen correctly.

THE TAKEAWAY

Two silver medals and no bronze is RANK. Two silvers and then a bronze is DENSE_RANK. Deciding which your report means takes ten seconds and is never written down.

All four, one query

```
1 SELECT product_key, units,  
2         ROW_NUMBER() OVER w AS rn,  
3         RANK()      OVER w AS rnk,  
4         DENSE_RANK() OVER w AS dense,  
5         NTILE(4)     OVER w AS quartile  
6 FROM vw_product_units_aug  
7 WINDOW w AS (ORDER BY units DESC);
```

7 One window definition, four functions. They cannot disagree about the ordering, only about the ties.

THE TAKEAWAY

Naming the window once is not only shorter. It makes it structurally impossible for the four columns to be ranking along different orders.

ROW_NUMBER must return 1 to n. Which means it **has** to break every tie, and it breaks them by choosing.

What that forces on you

- ◆ **An incomplete ordering makes ROW_NUMBER non-deterministic.** Not approximately. The result may differ between runs on frozen data.
- ◆ **And you cannot see it in the output.** The numbers still run 1 to n, looking exactly as they should.
- ◆ **So every ROW_NUMBER needs a unique tiebreaker inside its OVER clause.** Not in the query's ORDER BY, which is a different clause.
- ◆ **RANK and DENSE_RANK are immune to this,** because they are allowed to say two rows are equal, and so they do.

THE TAKEAWAY

This is the one function in the family that can silently choose for you. It is also the one used for deduplication, which is where the choice becomes a deletion.

Two things called duplicates

- ◆ **An exact duplicate is byte-identical.** Same values in every column, usually from a file loaded twice.
- ◆ **A logical duplicate shares a key and differs elsewhere.** Two versions of one customer, with different addresses.
- ◆ **DISTINCT removes the first kind and cannot touch the second.** It compares whole rows, so any difference keeps both.
- ◆ **Nearly every real deduplication is the second kind,** which is why ROW_NUMBER exists and DISTINCT is not enough.

THE TAKEAWAY

Ask which kind you have before choosing a tool. If the rows differ at all, you are not removing duplicates, you are choosing a winner.

Count them before you remove them

```
1 SELECT COUNT(*) AS total_rows,  
2 COUNT(DISTINCT customer_durable_key) AS distinct_keys,  
3 COUNT(*)  
4 - COUNT(DISTINCT customer_durable_key) AS to_remove  
5 FROM dim_customer;
```

- 4 The number of rows a correct dedup should delete. Write it down before running the dedup, and compare afterwards.

THE TAKEAWAY

Three counts, one query, and now the deduplication has an expected answer. Without it, any row count out is as plausible as any other.

The deduplication pattern

```
1 WITH ranked AS (  
2     SELECT c.*,  
3           ROW_NUMBER() OVER (  
4               PARTITION BY customer_durable_key  
5               ORDER BY valid_from DESC, customer_key DESC  
6           ) AS rn  
7     FROM dim_customer c  
8 )  
9 SELECT * FROM ranked WHERE rn = 1;
```

- 4 The partition names what a duplicate is. Two rows sharing this are candidates to collapse.
- 5 The ordering names which one survives. The second column is the tiebreaker, and it is not optional.

THE TAKEAWAY

The most-written window query in the world, and it encodes two business decisions on two lines. Most reviews check neither.

The two decisions a dedup makes

- ◆ **What counts as a duplicate.** That is PARTITION BY, and it has to match the real uniqueness grain of the data.
- ◆ **Which duplicate wins.** That is ORDER BY, and it must end in something unique or the winner is arbitrary.
- ◆ **Get the first wrong and you delete real rows,** not duplicates. The row count drops and the query still succeeds.
- ◆ **Get the second wrong and the surviving row changes between runs,** which is scenario one of module two.

THE TAKEAWAY

Two questions, both answerable in English before any SQL is written. 'A duplicate is two rows with the same X, and the one we keep is the one with the latest Y.'

The same dedup, two dialects

QUALIFY

Redshift, Snowflake, DuckDB

```

1 SELECT *
2   FROM dim_customer
3  QUALIFY ROW_NUMBER() OVER (
4          PARTITION BY customer_durable_key
5          ORDER BY valid_from DESC,
6                  customer_key DESC
7          ) = 1;

```

One query block. The window is never projected, so no stray rn column reaches the output.

CTE

Postgres, SQL Server, everywhere

```

1 WITH ranked AS (
2   SELECT *, ROW_NUMBER() OVER (
3       PARTITION BY customer_durable_key
4       ORDER BY valid_from DESC,
5               customer_key DESC
6   ) AS rn
7   FROM dim_customer
8 )
9 SELECT * FROM ranked WHERE rn = 1;

```

Portable, and the rn column is in the output unless you list the columns instead of using a star.

THE TIEBREAKER

Identical meaning and identical plans. The difference is nine lines against seven, and one of them leaks a column into SELECT star.

ROW_NUMBER with no ordering at all

- ◆ **Some engines permit it and number the rows arbitrarily.** Postgres and Redshift both accept an empty OVER here.
- ◆ **Others reject it**, because a row number without a sequence has no defined meaning to give you.
- ◆ **It is legitimate for one job: assigning surrogate keys**, where you want distinct integers and genuinely do not care which row gets which.
- ◆ **It is never legitimate for choosing a survivor.** That is a coin toss written in SQL, run nightly.

THE TAKEAWAY

The engines that reject it are doing you a favour. The ones that accept it will happily let a deduplication pick at random for years.

RANK skips after a tie. That is not a defect,
and somebody will report it as one.

Where the gaps come from

```
1 SELECT product_key, units,  
2         RANK()      OVER (ORDER BY units DESC) AS rnk,  
3         DENSE_RANK() OVER (ORDER BY units DESC) AS dense  
4 FROM vw_product_units_aug  
5 ORDER BY units DESC, product_key;
```

- 2 RANK counts how many rows are strictly better, plus one. Three rows tied at fifth means the next is eighth.
- 3 DENSE_RANK counts how many distinct values are better, plus one. It cannot skip, because values cannot be skipped.

THE TAKEAWAY

The definitions explain the behaviour. RANK counts rows. DENSE_RANK counts distinct values. Everything about gaps follows from which of those two things is being counted.

Which one, and when

WHAT THE REPORT MEANS	THE FUNCTION
Every row needs its own position	ROW_NUMBER, with a tiebreaker
Equal performance should read as equal	RANK or DENSE_RANK
Position should reflect how many beat you	RANK
Position should reflect how many levels beat you	DENSE_RANK
I need to delete duplicates	ROW_NUMBER, always
I need buckets of roughly equal size	NTILE

THE TAKEAWAY

Only one row in this table has no alternative. Deduplication requires exactly one survivor, and only ROW_NUMBER guarantees exactly one.

Rank within, and rank overall

```
1 SELECT s.region_key, f.product_key,  
2         RANK() OVER (PARTITION BY s.region_key  
3                       ORDER BY SUM(f.net_amount) DESC) AS rank_in_region,  
4         RANK() OVER (ORDER BY SUM(f.net_amount) DESC) AS rank_national  
5 FROM fact_sales_line f  
6 JOIN dim_store s ON s.store_key = f.store_key  
7 GROUP BY s.region_key, f.product_key;
```

4 No partition, so the whole result is one group. Seven regional number ones, and one national number one.

THE TAKEAWAY

A product ranked first in its region and ninetieth nationally is the interesting row. Neither number alone tells you that, and both come from one pass.

One rank, several uses

- ◆ **Compute the rank once, in a CTE, then filter it several ways.** Top three, bottom three, and the middle, from one sort.
- ◆ **Writing three queries with three rankings does the sort three times**, and they can disagree if anybody edits one.
- ◆ **Bottom N is the same rank, counted from the other end.** Either reverse the ordering or filter on the count minus the rank.
- ◆ **COUNT(*) OVER the same partition gives you the denominator**, so 'rank 3 of 47' needs no second query.

THE TAKEAWAY

A rank and a partition count are computed from the same sort. Asking for both together is nearly free, and asking separately is not.

A merchandising report ranks products within each category by units sold, using RANK. The published list for one category reads 1, 2, 2, 4, 5, 5, 5, 8 and continues with similar gaps throughout.

A category manager raises a defect saying that products at ranks 3, 6 and 7 are missing from the report, and asks for the pipeline to be rerun. The row count matches the number of products in the category exactly.

What is the correct response?

- ☐ A Nothing is missing. RANK skips after ties by design, and DENSE_RANK is the fix only if consecutive numbering is the actual requirement.
- ☐ B Switch to ROW_NUMBER, which never skips a number and so cannot produce this complaint again.
- ☐ C The ties indicate duplicated sales lines, which should be removed before the ranking is computed.
- ☐ D Add a tiebreaker to the ORDER BY so that no two products can tie, which removes the gaps at source.

What is the correct response?

- A

 Nothing is missing
The row count matching the product count proves it. Gaps are in the numbering, not in the data, and the requirement decides whether that matters.
- B

 Switch to ROW_NUMBER
Removes the complaint and the information with it. Products selling identical units would then be shown at different ranks, chosen arbitrarily.
- C

 Duplicated sales lines
Worth ruling out, and the row count rules it out. Products genuinely do sell identical unit counts, particularly at low volumes.
- D

 Add a tiebreaker
Makes rank behave like row number by removing every tie. It answers the complaint by changing what the report claims.

THE SIGNAL

"The row count matches the number of products exactly." Nothing has been dropped. The gaps are in the numbering, which is a labelling decision rather than a data one.

'The top three' is **ambiguous** whenever three rows might tie for third.

Top three per region

```
1 WITH ranked AS (  
2     SELECT s.region_key, f.product_key,  
3           SUM(f.net_amount) AS sales,  
4           ROW_NUMBER() OVER (  
5               PARTITION BY s.region_key  
6               ORDER BY SUM(f.net_amount) DESC, f.product_key  
7           ) AS rn  
8     FROM fact_sales_line f  
9     JOIN dim_store s ON s.store_key = f.store_key  
10    GROUP BY s.region_key, f.product_key  
11 )  
12 SELECT * FROM ranked WHERE rn <= 3;
```

6 A window over an aggregate, from module one. The ranking runs at step five, on the grouped rows.

12 Exactly 21 rows out. Three per region, whatever the data does, because ROW_NUMBER cannot tie.

THE TAKEAWAY

Exactly three per region is a promise this query can keep. It keeps it by choosing between equal products, and product_key is the tiebreaker doing the choosing.

Top three, two readings

`ROW_NUMBER <= 3`

Exactly three rows

- Always three per group
- Ties broken arbitrarily or by you
- Predictable output size
- A row may be dropped unfairly

`RANK <= 3`

At least three rows

- Three or more per group
- Ties all kept together
- Output size depends on data
- No row is dropped unfairly

THE TIEBREAKER

Downstream consumers usually care about one of these two properties very much. Ask which: a fixed row count, or no arbitrary exclusions.

Dense rank makes a third reading

- ◆ **DENSE_RANK <= 3** gives the top three performance levels, however many products occupy them.
- ◆ **Which can be a lot of rows.** At low unit volumes, dozens of products share a value, and three levels might be ninety rows.
- ◆ **It is the right reading for price bands, star ratings and scores,** where the value is a category rather than a measurement.
- ◆ **Three readings, three functions, one English phrase.** That is the problem, and the phrase is not going to improve.

THE TAKEAWAY

Top three rows, top three including ties, top three levels. Say which one out loud in the ticket, because the SQL will not say it for you.

Top three including ties

```
1 WITH ranked AS (  
2     SELECT category_key, product_key, units,  
3           RANK() OVER (PARTITION BY category_key  
4                       ORDER BY units DESC) AS rnk  
5     FROM vw_product_units_aug  
6 )  
7 SELECT * FROM ranked  
8 WHERE rnk <= 3  
9 ORDER BY category_key, rnk, product_key;
```

8 Three or more rows per category. Four products tied at first place returns four rows and no second or third at all.

THE TAKEAWAY

The output size now depends on the data, which matters to whoever consumes it. A fixed-size slot on a dashboard cannot take this query.

The second highest, per group

```
1 WITH ranked AS (  
2     SELECT store_key, product_key, units,  
3           DENSE_RANK() OVER (PARTITION BY store_key  
4                               ORDER BY units DESC) AS dr  
5     FROM vw_store_product_units  
6 )  
7 SELECT store_key, product_key, units  
8     FROM ranked  
9 WHERE dr = 2;
```

- 3 DENSE_RANK, because second highest means the second distinct value, not the second row.
- 9 A store where every product sold the same amount has no rank two at all, and returns nothing.

THE TAKEAWAY

Second highest value is DENSE_RANK. Second row is ROW_NUMBER. The English phrase does not distinguish them and the answers differ whenever anything ties.

Why that question is asked

- ◆ **It has four defensible answers**, and the differences between them are entirely about ties.
- ◆ **The wrong instinct is a subquery with MAX below MAX.** It works on clean data and breaks on the first tie.
- ◆ **The right answer starts with a question.** Second highest value, or second row, and what if there is no second?
- ◆ **Which is what the question is actually testing.** Not the syntax, which is one line either way.

THE TAKEAWAY

Every ranking problem worth solving reduces to a decision about ties. If the interview version feels like a trick, so does the production version.

If the partition does not match the real uniqueness grain, a dedup **deletes real rows**.

One partition column short

```
1 WITH ranked AS (  
2     SELECT p.*,  
3         ROW_NUMBER() OVER (  
4             PARTITION BY product_key  
5             ORDER BY effective_from DESC  
6         ) AS rn  
7     FROM fact_price_change p  
8 )  
9 SELECT * FROM ranked WHERE rn = 1;
```

- 4 A price at Kestrel is identified by product and store. Partitioning by product alone treats 412 store prices as duplicates of each other.

THE TAKEAWAY

41 million rows in, and far fewer out than anybody expected. Every store-specific price for a product was collapsed into whichever one happened to be latest.

Kestrel's price staging job deduplicates fact_price_change with ROW_NUMBER partitioned by product_key, ordered by effective_from descending, keeping row one. The table holds national prices with a null store_key and store-level overrides with a real one.

Margin reporting for 38 stores has been wrong for six weeks. Those stores all carry local price overrides. The job runs clean, the row count drops as expected, and no error has ever been raised.

What is the defect?

- ☐ A Use RANK rather than ROW_NUMBER, so that tied rows survive instead of being discarded.
- ☐ B Add store_key to the ORDER BY, so that the newest store-level price wins over the national one.
- ☐ C Deduplicate only the national prices, and pass the store-level overrides through untouched.
- ☐ D The partition is the wrong grain. A price is identified by product and store, so both belong in PARTITION BY.

◆ PAUSE AND DECIDE

What is the defect?

A Use RANK instead

Keeps tied rows and does nothing here, because the rows are not tied. They have different effective dates and different stores.

C Only deduplicate national prices

Assumes store overrides never duplicate, and they do. It half-fixes the deletion and leaves the duplicates it was meant to remove.

B Add store_key to the ORDER BY

Changes which single row survives per product. One row per product is the defect, so a better choice of survivor does not help.

D Wrong partition grain

Every store's price for a product was treated as a duplicate of every other. 411 real rows deleted per product, silently.

THE SIGNAL

"The row count drops as expected." A deduplication step is meant to drop rows, so the symptom of this failure is indistinguishable from success.

Four ways to deduplicate

METHOD	WHEN IT IS THE RIGHT ONE
ROW_NUMBER then filter	You need whole rows, and a specific one wins. The default.
GROUP BY with MIN or MAX	You need a few columns, not whole rows. Cheaper, and it cannot pick a row.
DISTINCT	The rows are byte-identical. Otherwise it is not deduplication at all.
DISTINCT ON	Postgres only. Shortest to write, and it will not run anywhere else.

THE TAKEAWAY

The second row is the one people forget. If you want the latest date per product and nothing else, MAX is one pass and no sort of the whole table.

Why MIN is not the same as ROW_NUMBER

- ◆ **MAX(effective_from)** gives you the date. It does not give you the row that had it.
- ◆ **So MAX over several columns mixes rows.** The latest date and the highest price may belong to two different records.
- ◆ **That produces a row that never existed**, assembled from parts, and every value in it is real.
- ◆ **ROW_NUMBER keeps a row intact.** When the columns have to be consistent with each other, that is the only safe option.

THE TAKEAWAY

A Frankenstein row passes every column-level check, because every column came from real data. It is only wrong as a whole.

Assert what the dedup did

```
1 SELECT COUNT(*) AS rows_out,  
2         COUNT(*) FILTER (  
3           WHERE rn > 1) AS should_be_zero,  
4         COUNT(*) - COUNT(DISTINCT product_key  
5                           || ':' || COALESCE(store_key::text, 'N'))  
6         AS still_duplicated  
7 FROM deduped;
```

6 Both figures must be zero. If either is not, the partition key does not match the grain you believe in.

THE TAKEAWAY

An assertion that runs in seconds and would have caught scenario two on its first night. It is the difference between a job that works and a job you know works.

Three assertions worth the space

- ◆ **Rows out equals distinct keys in.** Computed before the dedup ran, compared after it did.
- ◆ **No key appears twice in the output.** A grouped count with HAVING, expected to return nothing.
- ◆ **The measure total is unchanged, where it should be.** Deduplicating versions should not move a sum of amounts.
- ◆ **Together they cover both failure directions,** which are deleting too much and deleting too little.

THE TAKEAWAY

The third one is the cheapest and the most often skipped. If a dedup changed a total it was not supposed to touch, something was not a duplicate.

Deduplicate here, or upstream

IN THE WAREHOUSE

Where you can see it

- Visible, testable, versioned
- Runs on every load, forever
- Costs a sort every night
- Hides a defect from the source team

AT THE SOURCE

Where it came from

- Fixes the cause, not the symptom
- Needs another team and a quarter
- No recurring cost at all
- May simply not be possible

THE TIEBREAKER

Doing it in the warehouse is almost always correct as a first move and almost never correct as a final one. The mistake is forgetting there was a second move.

NTILE divides by **row count**, not by value.
Equal rows can land in different buckets.

Buckets that are equal in count

```
1 SELECT member_durable_key, annual_spend,  
2         NTILE(10) OVER (  
3         ORDER BY annual_spend DESC  
4         ) AS spend_decile  
5 FROM vw_member_annual_spend;
```

- 2 1.2 million members become ten buckets of 120,000 each, regardless of what the spend values do.
- 3 No tiebreaker, so members on identical spend are assigned to buckets in whatever order the plan produced.

THE TAKEAWAY

Ten equal groups is a defensible definition of a decile. Ten value bands is a different one, and the report will not say which it used.

What NTILE does at the edges

- ◆ **When rows do not divide evenly, the earlier buckets get the extra rows.** Bucket one is the largest, by at most one.
- ◆ **A partition with fewer rows than buckets produces fewer buckets.** NTILE(10) over four rows gives buckets one to four, and nothing else.
- ◆ **So a small partition can have no bucket ten at all,** and a report joining on bucket number will silently lose it.
- ◆ **Check the smallest partition before choosing the bucket count.** The largest one is never the problem.

THE TAKEAWAY

Empty buckets do not appear as rows with zero. They do not appear at all, because there was no row on which to compute one.

Kestrel's loyalty team assigns members to spend deciles with NTILE(10) over annual spend, and offers a benefit to deciles one and two. The decile is recomputed monthly across all 1.2 million members.

A member complains that their neighbour, with identical annual spend to the pound, received the benefit and they did not. The loyalty team confirm both figures are the same and both members were processed in the same run.

What should the team change?

- ☐ A Nothing. NTILE is the standard way to compute deciles, and equal-sized groups is what a decile means.
- ☐ B Use ROW_NUMBER divided by the total count, which produces the same buckets more transparently.
- ☐ C NTILE splits ties across buckets, so eligibility must be defined by a spend threshold rather than a bucket number.
- ☐ D Use RANK and take the top twenty percent, which avoids buckets and therefore avoids the problem.

◆ PAUSE AND DECIDE

What should the team change?

- A

 Nothing, NTILE is standard
Defensible for analysis and not for entitlement. Once a bucket decides who gets a benefit, an arbitrary boundary becomes a fairness problem.
- B

 ROW_NUMBER over a count
Reproduces NTILE exactly, including the arbitrary tie splitting. It makes the mechanism visible without changing it.
- C

 Define eligibility by value
Two members with equal spend get equal treatment, by construction. The bucket can stay for analysis, and it stops deciding anything.
- D

 RANK and top twenty percent
RANK keeps ties together, which helps, and twenty percent of rows still cuts somewhere. The boundary moves rather than disappearing.

THE SIGNAL

"Identical annual spend to the pound." When a customer can compare themselves to another customer, an arbitrary tiebreak stops being a technical detail.

Two more ranking functions, briefly

- ◆ **PERCENT_RANK** is the rank expressed as a fraction. Rank minus one, divided by row count minus one, so it runs 0 to 1.
- ◆ **CUME_DIST** is the share of rows at this value or better. It never returns zero, because the current row always counts.
- ◆ **Both handle ties the way RANK does**, giving tied rows the same value, which is why they beat NTILE for fairness.
- ◆ **Module nine takes them properly**, alongside the percentile functions, which are a different thing again.

THE TAKEAWAY

If NTILE is failing you because of ties, these two are usually the answer rather than a bigger bucket count.

Ranking over a nullable column

- ◆ **Nulls rank too.** They are not skipped, and they do not come last unless the engine decides they should.
- ◆ **On Redshift and Postgres, null is greatest.** So ORDER BY ... DESC gives every null row rank one.
- ◆ **Which makes a dedup keep the null row,** because ROW_NUMBER equals one is exactly where it landed.
- ◆ **Write NULLS LAST inside the OVER clause,** or filter the nulls out before ranking. Both are one line.

THE TAKEAWAY

Module two's default arrives inside a ranking function and picks your survivor. A dedup ordered by a nullable date will systematically keep the rows with no date.

The survivor you did not intend

```
1 ROW_NUMBER() OVER (  
2   PARTITION BY product_key, store_key  
3   ORDER BY effective_from DESC NULLS LAST,  
4             price_change_key DESC  
5 )
```

- 3 Without NULLS LAST, rows with no effective date win every partition they appear in.
- 4 And the tiebreaker, so that two rows with the same date still produce a defined winner.

THE TAKEAWAY

Two clauses, both one line, both routinely omitted. Together they turn a dedup from something that usually works into something that always does.

What ranking costs

- ◆ **Every ranking function needs a sort of each partition.** There is no way to rank without ordering first.
- ◆ **A dedup therefore sorts the whole table,** even when only a fraction of the rows are duplicated at all.
- ◆ **Find the duplicated keys first, then rank only those.** Two cheap passes can beat one enormous sort.
- ◆ **Engines optimise top-N specially.** A small filter on ROW_NUMBER can often avoid a full sort, and a dedup cannot.

THE TAKEAWAY

Top three per region and deduplication look like the same query. One touches three rows per partition and the other touches all of them, and the plans reflect that.

Paging without drift

```
1 WITH page AS (  
2     SELECT product_key, units,  
3           ROW_NUMBER() OVER (  
4             ORDER BY units DESC, product_key  
5           ) AS rn  
6     FROM vw_product_units_aug  
7 )  
8 SELECT * FROM page  
9 WHERE rn BETWEEN 101 AND 150;
```

4 A total order, so every page has defined membership and page 3 is page 3 on every run.

THE TAKEAWAY

OFFSET 100 LIMIT 50 gives the same rows only if the ordering is total. Without a tiebreaker, rows can appear on two pages or on none.

Why paged reports drift

- ◆ **Each page is a separate query.** The engine is under no obligation to order ties the same way twice.
- ◆ **So a row can appear on page two and again on page three**, while another appears on neither. The counts still look right.
- ◆ **A total order fixes it completely**, and costs one column in the ORDER BY.
- ◆ **Keyset paging is the scalable version.** Filter on the last row's key rather than counting rows to skip.

THE TAKEAWAY

Users report this as items disappearing from a list. It is one of the very few determinism bugs that somebody outside the data team will notice for you.

A rank is derived, not stored

- ◆ **Storing a rank freezes a relationship that keeps moving.** One new row changes the rank of everything below it.
- ◆ **And it is not additive.** You cannot combine yesterday's ranks with today's rows to get today's ranks.
- ◆ **Store the measure and compute the rank at read time.** Units sold is a fact. Rank three is an opinion about a moment.
- ◆ **If a rank must be kept, keep it with its date and its population,** because neither is recoverable afterwards.

THE TAKEAWAY

Ranks are the classic non-additive measure. Two stored ranks cannot be added, averaged, or compared across populations, and all three get attempted.

A nightly staging job deduplicates a 2.41 billion row landing table with ROW_NUMBER partitioned by the natural key. It takes just under four hours and spills heavily to disk. About 0.3% of rows are duplicated, roughly seven million.

The job is on the critical path for the morning refresh and has twice overrun into business hours. The team has been asked to bring it under an hour without changing the output.

What is the most effective change?

- A** Increase the cluster size, since the job spills to disk and is clearly memory-bound rather than algorithmically wrong.
- B** Find the duplicated keys with a grouped count, rank only those rows, and union them back with the untouched majority.
- C** Replace ROW_NUMBER with DISTINCT, which removes duplicates without sorting into partitions.
- D** Replace it with an anti-join against a table of keys to discard, built in a prior step.

◆ PAUSE AND DECIDE

What is the most effective change?

A Increase the cluster

Reduces the spilling and still sorts 2.41 billion rows to correct seven million. It buys time against a cost that grows every night.

C Use DISTINCT instead

Only equivalent if duplicate rows are byte-identical. If they differ in any column, DISTINCT keeps both and the output changes.

B Rank only the duplicated keys

A grouped count is cheap and parallel. The sort then covers 0.3% of the table, and 99.7% of rows are passed through untouched.

D Anti-join a discard list

Sound, and it needs the keys ranked to build the list, so the expensive step has moved rather than gone.

THE SIGNAL

"0.3% of rows are duplicated." The work is proportional to the whole table and the problem is proportional to a rounding error. That gap is the entire optimisation.

Choosing, in four questions

01

Purpose

Deleting rows, or labelling them?
Deleting is always ROW_NUMBER.

02

Ties

Should equal rows read as equal? If yes, RANK or DENSE_RANK.

03

Gaps

Should the numbers be consecutive?
DENSE_RANK if yes, RANK if not.

04

Order

Is the ordering unique, and where do nulls go?
Fix both before shipping.

THE TAKEAWAY

The fourth step applies whichever function you picked, and it is the only one that can make yesterday's answer differ from today's.

Engine differences worth knowing

BEHAVIOUR	WHERE IT DIFFERS
ROW_NUMBER with no ORDER BY	Accepted by some engines, rejected by others.
QUALIFY	Redshift, Snowflake, Databricks, BigQuery, DuckDB. Not Postgres.
DISTINCT ON	Postgres. Nothing else in common use.
NULLS FIRST or LAST	Postgres family yes. MySQL and SQL Server have no syntax for it.
Default NULL rank	Greatest in the Postgres family. Smallest in MySQL and SQL Server.

THE TAKEAWAY

The last two rows change an answer rather than raising a syntax error. A dedup moved between engines can silently pick a different survivor.

The confusions, settled

CONFUSION

RANK or DENSE_RANK

RANK or ROW_NUMBER

Why are numbers missing

Why did the survivor change

Why did rows disappear

Why are equal rows in different buckets

THE RESOLUTION

RANK counts rows that beat you. DENSE_RANK counts values.

ROW_NUMBER cannot tie. Everything else follows from that.

RANK skipped after a tie. Nothing was dropped.

The ordering was not unique inside the partition.

The partition was coarser than the real uniqueness grain.

NTILE divides by count, not by value.

THE TAKEAWAY

The bottom three rows are defects. The top three are people meeting a convention they were not told about, and they cost almost as much time.

Five habits for this whole family

- ◆ **End every ranking ORDER BY with something unique.** No exceptions, and it costs one column.
- ◆ **Write NULLS LAST whenever the ordering column is nullable.** Or exclude the nulls deliberately first.
- ◆ **Say the partition out loud as a sentence.** 'A duplicate is two rows with the same ...' and see whether it is true.
- ◆ **Predict the row count before you run a dedup,** then check it afterwards.
- ◆ **Ask which reading of 'top three' was meant,** before choosing between three functions that all look right.

THE TAKEAWAY

None of these is about knowing the functions. All five are about the two lines underneath them, which is where every defect in this module lived.

Ranking needs no frame. **Module five** is about everything that does.